

VarBlockSpMM: Shape-Aware CUDA Execution for Packed Variable-Block Sparse-Dense Products

Cohen Brunner Thomas
Independent Research Project

6 September 2026

Abstract

VarBlockSpMM achieves a geometric speedup of 1.139 over the fastest tested library in a 128-case FP32 sparse-dense comparison on one RTX 5060 Ti, including MAGMA variable-size batched GEMM. It wins 105 cases. At dense panel widths 8, 16, 32, and 64, the respective ratios are 1.060, 1.045, 1.088, and 1.398, making width 64 the strongest measured regime. The implementation reads packed variable blocks directly, avoiding scalar expansion, retiling, and batched compositions. One thread array owns each block row and writes its output once, with no execution workspace. Panel width selects the staging and accumulation policy.

The evaluation measures when these choices outweigh library execution and representation costs. A purpose-built variable-order discontinuous Galerkin transport trace runs 1.439 times faster than the fastest library at 4,096 elements, 64 fields, and 128 dependent steps. However, a specialized fused kernel exploiting repeated operators beats direct execution by 1.359 on that trace. BSR8 also wins covariance steady state, and compact CSR wins the published matrices with artificial partitions. Narrow-panel gains are smaller and vary by configuration. The results support direct execution for the measured packed-block regimes, with independent application adoption and hardware portability still untested.

1 Introduction

Applications with dense variable blocks face a representation choice when using sparse GPU libraries. Scalar CSR expands block metadata to individual entries. Fixed-block BSR subdivides the original blocks and may require a new value layout. Grouped GEMM can retain block shapes, but overlapping contributions require separate calls or a reduction. VarBlockSpMM executes the packed product directly, so an application can retain its block layout and avoid these conversion steps.

This is useful when direct execution is faster, when scalar metadata is costly, or when a matrix is used too few times to repay library conversion and setup. We measure these benefits separately. The intended input has dense stored blocks. Artificially grouping scattered scalar nonzeros can instead waste storage and arithmetic, as the published-matrix controls show.

We study $C = AB$, where $A \in \mathbb{R}^{M \times K}$ is a packed variable-block sparse row matrix and $B \in \mathbb{R}^{K \times N}$ and $C \in \mathbb{R}^{M \times N}$ are column-major dense panels. The supported block dimensions are 8, 16, $\dots$, 64, with independently chosen row and column partitions. Dense panel widths are $N \in \{8, 16, 32, 64\}$. The implementation uses FP32 values and accumulation for non-transpose multiplication with $\alpha = 1$ and $\beta = 0$.

The implementation specializes execution by panel width while retaining runtime block dimensions. One CTA owns each block row, shares the dense input tile, and writes the completed output once. This combines a small execution plan with direct access to the application’s packed values.

The contribution is a measured execution policy for this packed format. The organizing question is when reuse within a block row outweighs the staging and representation costs of achieving it. Panel width determines how many outputs can reuse each sparse value, while thread mapping and buffering determine the register, shared-memory, and synchronization costs. Matched experiments hold output ownership fixed to compare these choices. Library comparisons then test whether the resulting execution savings outweigh padding-free BSR8 subdivision or scalar CSR expansion.

This gives readers a way to evaluate the same tradeoff in another packed implementation: separate within-row reuse from conversion and startup costs, then measure both at the intended panel width and reuse count. The selected thresholds remain specific to the measured GPU. One CTA per block row, staging, and runtime block dimensions are established mechanisms. Fixed tiles with masking or several specialized kernels are also

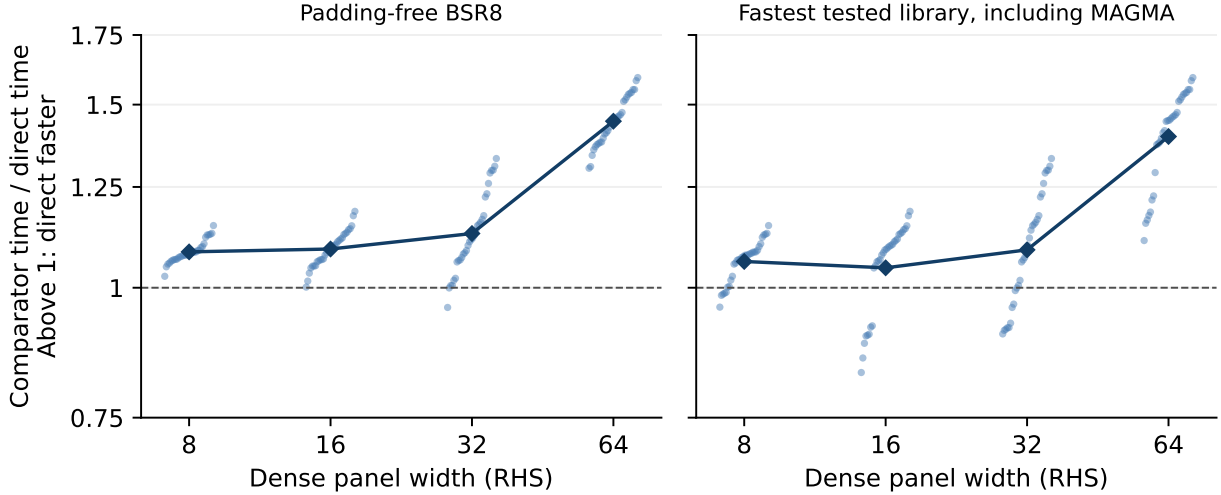


Figure 1: Direct execution by panel width on the 128-case core with MAGMA. Each point averages paired log ratios over three matrix seeds and three processes per seed, with twenty timing samples per process. Lines show geometric means across configurations. The right panel includes both MAGMA compositions and every applicable NVIDIA control. Horizontal offsets separate points. Ratios above one favor direct execution.

possible. Our contribution is their measured execution policy and performance boundary, not a new sparse storage model or an end-to-end application speedup.

The evaluation uses a purpose-built transport program, generated covariance blocks, and artificially partitioned published matrices. It establishes local performance regimes rather than adoption by an independent application. BSR8 wins covariance steady state, compact CSR wins the published-matrix controls, and a specialized fused kernel wins the transport comparison. Section 6 states the remaining external-validity limits.

Startup and storage measurements separately test the cost of adopting each representation, with raw measurements tied to verified builds. Figure 1 shows where the tested widths and workloads favor direct execution.

2 Related work

Variable blocking is established prior art. Vuduc and Moon studied variable and unaligned blocks for sparse matrix-vector multiplication [6]. OSKI defines variable block row storage through independent logical row and column partitions [7]. VarBlockSpMM uses this structural model for a dense multi-column right operand on CUDA.

GPU SpMM work has examined row assignment, load balance, instruction-level parallelism, and reuse. Yang, Buluç, and Owens describe row-splitting and merge-based approaches [1]. GE-SpMM caches sparse rows and merges warps to reduce redundant loads [8]. Tensor-core methods such as Okanovic et al. trade packing and preprocessing for dense matrix-instruction throughput [2]. These techniques motivate the present kernel. We compare the relevant MAGMA composition below, but do not compare implementations of these other research kernels.

SMaT [2] is not measured in this FP32 evaluation. An application-level comparison could use different arithmetic if it passes the same output tolerances, with conversion and preprocessing charged at the stated reuse count. That experiment remains outside this study. BSR8 measurements do not establish the cost-benefit balance of SMaT’s execution path.

Hierarchical matrix construction motivates the packed-block product itself. Boukaram et al. describe a non-uniform batched BSR product implemented through slot-wise MAGMA non-uniform batched GEMM operations [5]. That work targets a much broader construction pipeline, and it reports leaf sizes of 64 to 256, whereas this implementation supports dimensions 8 through 64 in steps of eight. Only the 64 boundary overlaps those leaves. We therefore treat that study as background for the operation rather than as a workload this paper reproduces, and we do not claim a result for its pipeline.

The application evidence in this paper comes from a standalone variable-order discontinuous Galerkin transport program, described in Section 5.2. In that program the discretization supplies the block structure, an assembled

operator drives a sequence of dependent products, and every final coefficient is checked against repeated double-precision products and against the analytically translated wave. It is the only workload here with those three properties. Wide dependent traces are where direct execution helps most. A separate generated covariance near-field component, whose partitions come from a spatial cluster tree, tests a dense-block component drawn from the hierarchical setting. It excludes far-field compression and the complete construction pipeline, and BSR8 wins its steady-state comparison, so we report it as a boundary rather than as support for the method. NVIDIA provides scalar and fixed-block SpMM through cuSPARSE [3], and per-group GEMM dimensions through grouped cuBLAS [4]. These form the library controls. Asynchronous global-to-shared copies can overlap tile movement and arithmetic [9]. These mechanisms are established techniques. We combine and evaluate them for this particular variable-block operation.

3 Representation and kernel

Let block-row boundaries be $r_0 < \dots < r_R$, and block-column boundaries be $c_0 < \dots < c_Q$. Block (i, j) has height $h_i = r_{i+1} - r_i$ and width $w_j = c_{j+1} - c_j$. Its column-major values are packed without rounding its dimensions to a global block size, and its contribution is

$$C[r_i : r_{i+1}, :] += A_{ij}B[c_j : c_{j+1}, :].$$

Block-row pointers and column indices locate the blocks. Scalar offsets and value offsets are 64-bit. Block indices remain 32-bit. Host validation checks monotonic offsets, dimensions, index bounds, and payload sizes. A matrix owns its GPU storage and row classifications. Execution plans hold non-owning views, so the matrix must outlive its plans.

3.1 Output ownership and accumulation

One cooperative thread array, or CTA, owns one block row at every supported RHS width. Threads outside the actual row height do no output work. Each active thread accumulates all contributions to its outputs and writes them once. CTAs never update the same scalar, so execution needs neither output atomics nor partial-output reduction storage. Work imbalance between rows can still limit performance.

RHS 8 and 16 use 64-thread CTAs, with each active thread owning all RHS columns of one scalar row. RHS 32 assigns eight output columns per thread, and RHS 64 assigns sixteen. Multiple accumulators let each loaded sparse value feed independent multiply-add operations. Their register cost must be balanced against the number of resident CTAs and active threads.

Ownership and tile sequence for one block row

Ownership	One CTA writes the entire output panel of block row i .
Initialization	Each thread sets its owned accumulators to zero.
First tile	Load the first reusable B tile into shared memory.
Pipeline step	Initiate the next tile copy into the alternate buffer, then compute the current block contribution.
Boundary	Wait for copies and synchronize the CTA before consuming the next tile or reusing a buffer.
Completion	Write each owned output once, including zero for an empty row.

Figure 2: The tall-row RHS-32 algorithm with its alternating two-buffer schedule. All supported widths retain disjoint output ownership.

3.2 Full-panel accumulation for RHS 8 and 16

The narrow-panel kernel launches one 64-thread CTA per block row. Thread $t < h_i$ owns $C[r_i + t, 0 : N]$, with N FP32 accumulators. For each stored block, all 64 threads cooperatively load its $w_j \times N$ input tile into shared memory. A barrier makes that tile visible before use. Each active thread then loads one sparse value at a time and reuses it across all N columns. A second barrier protects shared-memory reuse when another block follows.

The shared leading dimension is 65 floats. The RHS-8 and RHS-16 kernels allocate 2,080 and 4,160 shared-memory bytes per CTA, respectively. Since the largest supported row has 64 scalars, one CTA covers the entire panel without a row-size threshold or a second output tile. Empty rows write zero.

The narrow policy was selected on development seed 4 at 256, 1,024, and 4,096 block rows, degrees 2 and 16, and all four shape distributions. We compared staged kernels with different thread and accumulator counts and warp-reduction candidates, checking every output against a full CPU reference. The final policy uses full-panel accumulation and staging at both widths. Evaluation seeds are separate from this development seed. The matched campaign also measures the previous narrow policy in the same executable and process, so its improvement does not depend on comparing archived times.

3.3 Staging and dispatch

At every supported width, each local output row consumes the same dense input tile for a stored block. Cooperative staging loads this tile once per CTA and reuses it across local rows. A shared leading dimension of 65 floats reduces power-of-two bank-alignment conflicts. The synchronous RHS-32 and RHS-64 kernels use 8,320 and 16,640 shared-memory bytes per CTA, respectively. The narrow kernels use the smaller CTAs described above. RHS 32 and 64 retain their width-specific staging and buffering policies.

For RHS 32, rows of at most 16 scalars use a 128-thread single-buffer CTA. Taller rows use a 256-thread CTA with two tiles, consuming 16,640 shared-memory bytes. The next input tile is prefetched before arithmetic on the current tile. Pipeline waits and CTA barriers protect visibility and buffer reuse. A mixed-height matrix is split into two launches only when its mean block degree is at least eight. Homogeneous matrices need only the applicable launch. At lower mean degree, the mixed matrix uses the unsplit policy.

Double buffering is not enabled for RHS 64. A development experiment increased shared allocation to about 32.5 KiB per CTA and regressed a representative median from 2.891 to 4.419 ms. This observation motivated a width-specific policy rather than a universal pipeline. All dispatch thresholds were selected from development sweeps and held fixed for the main library comparison.

4 Experimental design

The headline comparison uses the 128-configuration core grid, matrix seeds 2, 3, and 5, and three fresh processes per seed. Each process records 20 samples per method, separately for one product and eight queued products. The two modes use 2,304 fresh processes in total and are never pooled. The headline uses one-product samples, with equal weight for each seed, process, and configuration. Its comparator is the fastest applicable library within each process, including both MAGMA compositions. Dispatch thresholds remain fixed. Table 1 distinguishes this protocol from the other campaigns before their workload and comparator details.

Table 1: Main campaign designs. Each input instance and timing mode has three fresh processes. Samples are reduced to a median within each process. General and specialized transport comparisons use separate processes.

Campaign	Input variation	Samples per method and process
Headline libraries	128 configurations, seeds 2, 3, 5	20 per mode (batch 1 or 8)
Earlier synthetic	168 configurations, seed 1	20 single products
Published and native	Three published matrices and 12 generated geometries, four widths	20 single products
Matched mechanisms	32 configurations, seeds 1, 2, 3	20 single products
Controlled factors	24 configurations, seed 11	6 single products
Transport comparators	Six element-count and width pairs, fixed inputs	6 dependent traces

4.1 Persistent library controls

Scalar CSR. Synthetic dense blocks are expanded into scalar CSR once. Plans retain arrays, descriptors, handles, and workspace. We measure the default algorithm, explicit CSR algorithm 1 with preprocessing, algorithm 2, and algorithm 3 with preprocessing. All use FP32 compute, column-major panels, and the same non-transpose operation. Preprocessing and initial descriptor creation occur during setup or warmup, outside timing. NVIDIA documents preprocessing benefits for algorithms 1 and 3 [3].

The *CSR lower envelope* is the minimum GPU median among those four configurations within each process. Selecting on measured data favors this comparator. The envelope has no deployed tuning policy or charged tuning cost. It lets us check whether a direct-method win depends on the default library algorithm. Host ratios use a separate minimum over host medians, so they need not select the same algorithm.

Grouped cuBLAS. Blocks with equal height and width are grouped within each ordinal block slot. A slot contains at most one block from each row, so its GEMM outputs do not overlap. Slots execute sequentially. The first uses $\beta = 0$, and subsequent slots use $\beta = 1$. The handle, packed values, shape groups, and device pointer arrays persist.

The cached control refreshes input and output pointer arrays only when their base addresses change. It omits the full output clear when every row is nonempty. For matrices containing empty rows it retains the clear to initialize those rows. A separate changing-address experiment alternates between two allocated input/output pairs and includes pointer refreshes. It is paired with a direct method that alternates the same pairs. A stream-ordered CUDA kernel computes the input and output pointer arrays from persistent device offsets when needed. This avoids relying on the lifetime or staging behavior of temporary pageable host arrays. The first measured call reuses the final warmup addresses. The remaining 19 calls alternate addresses and refresh the grouped pointers. Changing-address timing includes these pointer-generation launches. The slot decomposition still incurs sequential GEMM calls and repeated output traffic, so it is a specific library composition rather than an oracle over possible grouped algorithms.

Fixed-block BSR. The uniform-32 synthetic subset is also measured with cuSPARSE BSR algorithm 1. Block values are converted to the required row-major block layout outside timing, while dense panels remain column-major. This gives a format-matched control where variable blocking is unnecessary.

Subdivided BSR8. Every supported dimension is divisible by eight. We therefore also subdivide each stored variable block into 8-by-8 blocks and measure cuSPARSE BSR algorithm 1 on every input. The scalar matrix and stored scalar count are unchanged, and no padding is introduced. A block of shape $h \times w$ becomes $(h/8)(w/8)$ fixed blocks, increasing block metadata while exposing a different scheduling and reuse tradeoff. Conversion to row-major BSR tiles occurs outside steady-state timing. The published scalar inputs retain their internal zeros in this control, whereas compact CSR removes them. BSR8 measures the effect of subdivision, while compact CSR measures the benefit of removing internal zeros.

4.2 Synthetic workloads

The core grid contains 128 configurations with 4,096 block rows and columns, degrees 2, 4, 8, 16, four panel widths, four size distributions, and two locality patterns. Uniform uses size 32. Low variance samples $\{24, 32, 40\}$, high variance samples $\{8, 16, \dots, 64\}$, and bimodal samples $\{8, 16, 48, 64\}$. Row and column dimensions are drawn independently. Every row in the core grid has exactly the requested degree. Columns are sampled without replacement and sorted. Random locality spans all block columns. Local locality samples cyclic offsets within plus or minus the requested degree. Stored values are uniform on $[-1, 1]$.

The extension adds 40 configurations. It evaluates 256, 1,024, and 4,096 block rows and columns for high-variance and bimodal sizes, random locality, all panel widths, and either constant degree 16 or the repeating degree pattern 0, 1, 16, 16. The latter has mean degree 8.25 and explicitly includes empty rows. It is constructed by retaining zero blocks, one block, or all blocks from successive rows of the degree-16 parent matrix. Eight constant-degree 4,096-row configurations already belong to the core grid. Thus the union contains 168 unique configurations.

The earlier NVIDIA-only synthetic campaign holds matrix seed 1 fixed and repeats each configuration in three fresh processes. The headline comparison uses the same 128-case core with seeds 2, 3, and 5, as specified in Table 1. The dense input contains independent pseudorandom uniform values on $[-1, 1]$, generated with seed 9123. Changing-address buffers contain identical values to isolate address-dependent work.

4.3 Published matrices and partitioning

We also test BCSSTK13, BCSSTK14, and BCSSTK15 from the Harwell-Boeing collection, downloaded through SuiteSparse [10]. They originate from fluid-flow eigenvalues, the Omni Coliseum roof, and an offshore platform module, respectively [11]. Their original orders are 2,003, 1,806, and 3,948.

Symmetric Matrix Market input is expanded, duplicates are combined, and explicit zeros are removed. Values are divided by the maximum absolute entry and converted to FP32. The rows and columns are padded only at the outer boundary to the next multiple of eight. In the original scalar ordering, row partitions cycle through 8, 16, 32, 64 and column partitions through 64, 32, 16, 8. The final partition is truncated to the

remaining multiple of eight. A block is stored if it contains a source nonzero. Other entries inside that block are represented as zeros.

This deterministic procedure preserves application-derived values and sparsity after normalization, but uses artificial partitions in place of native finite-element or hierarchical partitions. The measured compact-CSR controls retain only source nonzeros in the padded scalar domain, using explicit algorithms 1, 2, and 3 with preprocessing for 1 and 3. Their lower envelope is computed as above. Cached grouped cuBLAS uses the identical packed blocks as the direct method. This separates the cost of block fill from scalar sparse execution. All four panel widths and three fresh processes are retained. The partitions have 68, 61, and 132 block rows, respectively. RHS 8 and 16 use the same 64-thread staged kernel as the synthetic inputs.

4.4 Geometry-derived covariance near field

We construct the dense near-field component of a spatial covariance operator, following the bounding-box admissibility model described by Boukaram et al. [5]. Unlike grouping a pre-existing scalar sparse matrix, the spatial tree defines the dense blocks before matrix assembly.

Each input contains 2,048 or 8,192 points in three dimensions. Uniform geometry samples the unit cube. Clustered geometry places three quarters of the points in a clipped normal cloud with coordinate mean 0.3 and standard deviation 0.07, and leaves the remaining quarter uniform. Independent NumPy generator seeds 1, 2, and 3 give twelve matrices. For each matrix and panel width, three fresh processes repeat the same instance, giving 144 process observations.

A binary spatial tree splits along the longest bounding-box axis, using a stable coordinate sort and a midpoint rounded to a multiple of eight. A leaf stops at eight points, or at no more than 64 points when its bounding-box diameter is at most 0.18. Eight-point granularity is an explicit restriction of this application model. No points are padded, dropped, or grouped after scalar matrix assembly. The reordered leaf indices define both partitions. Uniform geometry at 2,048 points produces only size-eight leaves and is retained as a useful fixed-block control.

For leaf boxes s, t , let D denote Euclidean box diameter and dist the minimum Euclidean distance between boxes. We retain leaf pairs failing

$$\frac{D(s) + D(t)}{2} \leq 0.5 \text{dist}(s, t).$$

Each retained block stores the Gaussian covariance entries

$$A_{ij} = P^{-1/2} \exp(-\|x_i - x_j\|^2 / (2(0.1)^2)),$$

where P is the point count. The amplitude fixes the numerical scale across sizes. Other blocks are zero in this component. The far-field component of the full covariance operator is outside this experiment. The resulting near-field component need not itself be positive definite.

An independent scalar-row enumeration recomputes the covariance values and provides the compact-CSR CPU reference. Preparation checks every packed block against this reference exactly in FP32. Nonzero fill exceeds 99.99%, with a few FP32 underflows. This tests native spatial partitions within the specified model. It is not a production solver trace or an end-to-end hierarchical computation.

Some near-field inputs occupy a large fraction of the scalar matrix. We therefore include a persistent dense cuBLAS SGEMM control on all native and published-matrix inputs. It expands the compact scalar source to a zero-filled column-major dense matrix during startup, retains that matrix and a handle, and uses the same non-transpose operation and default cuBLAS math contract as the grouped control. Its output passes the same full-reference tolerances. Dense expansion is impractical at the main synthetic grid’s scalar dimensions and is not attempted there.

4.5 Matched mechanism experiments

The matched campaign crosses 256 and 1,024 block rows, degrees 2 and 16, high-variance and bimodal shapes, and four RHS widths with random locality. Each of the 32 configurations uses three matrix seeds and three fresh processes per seed, giving 288 processes. Both instance and process variation are therefore present, without treating timing trials as independent matrix samples.

For RHS 8, a complete $2 \times 2 \times 2$ factorial design varies linear versus eight-row output mapping, one versus two accumulators, and one versus two CTAs per block row. Every CTA has 256 threads. For wider panels, a global-load four-accumulator control and the selected wider accumulator count both use one 256-thread CTA per row. Cooperative staging is compared with direct loads using the same output mapping and CTA size.

Single and double buffering use the same indirect row order, CTA size, and accumulator count. At RHS 8 and 16, an additional pair compares global and shared input loads with identical 64-thread CTAs and full-panel accumulators. The previous narrow policy and the production dispatch are measured alongside these controls. Each variant is checked against a full double-accumulating CPU reference, rather than only sampled output probes.

These comparisons isolate the named policy choices at fixed input and launch dimensions. Register allocation, instruction scheduling, and active thread count can change as consequences of a choice. They are not held artificially constant. The factorial marginal effects average across the other two RHS-8 factors, and the retained per-variant timings expose interactions rather than assuming that speedups multiply.

4.6 Timing and correctness

The earlier synthetic, published, native, and matched campaigns give each method five warmups and 20 measured executions. Methods are randomly ordered within each process. Configuration and process-label jobs are shuffled before serial execution, with one GPU benchmark active at a time. Synthetic order uses seed 20260905, published-matrix order uses 20260906, and both supplementary campaigns use 20260907. Three process labels identify repeated observations of the same matrix, rather than three different matrix instances.

For single-product measurements, CUDA events bracket each execute call on the default stream, followed by an end-event synchronization. Queued headline samples bracket eight products and divide the elapsed time by eight. Synchronized host durations are also retained. The intervals measure execution calls, including any device idle time while the host submits later work. This matters particularly for library compositions with multiple calls, so the intervals do not isolate kernel-instruction time. Allocation, format conversion, initial input transfer, and correctness validation are excluded. Output buffers are poisoned before each method’s warmups, and correctness is checked both after warmup and after measurement. Every raw GPU and host duration, method position, command, executable checksum, and process status is saved. Medians use the mean of the two middle observations for an even sample count.

Device sources use FP32 multiply-add and accumulation with `--use_fast_math`, including flush-to-zero [15]. Accumulation order is unspecified. Subnormal intermediates, overflow, and cancellation can change agreement with a double-accumulated reference. The supplement gives worked edge cases and the input census. The reported workloads pass the checks below, which do not guarantee accuracy for arbitrary inputs or bound the timing effect of flush-to-zero.

The revision uses the RTX 5060 Ti with compute capability 12.0, CUDA runtime and driver API 13.4, Windows, MSVC 19.44, Release compilation, and native CUDA architecture. The Windows display driver reports version 32.0.16.1656. The GPU is not clock-locked or isolated from all operating-system activity. Per-process environment records accompany the synthetic timings.

The correctness suite covers both localities for every combination of four size distributions, four widths, and degrees 1, 4, 8, 16, giving 128 cases. Sixteen additional cases contain an empty first row. It tests explicit CSR algorithms, BSR32 and BSR8, cached pointer reuse, changed input and output addresses, and non-default streams. Deterministic tests cover all 64 block shapes at every RHS width, mixed empty rows, and completely empty matrices. NaN initialization before every comparison detects missing writes, and a deliberately empty operation must fail the comparison helper. Queued pointer changes are checked without intervening host synchronization. Every CUDA status in the test execution path is checked. Full output is compared with a CPU reference that accumulates the entire dot product in double before one final FP32 cast. Failure thresholds are maximum absolute error 5×10^{-4} or relative ℓ_2 error 5×10^{-5} , with NaN and infinity rejected.

Each timed synthetic matrix also receives 64 CPU output probes per method outside timing. Every application output is checked against a separate compact-CSR CPU reference, which also tests the packing transformation. Separate untimed comparison tests check every output for a uniform matrix with 4,194,304 stored values at RHS 64 and a high-variance matrix with 10,474,048 stored values at RHS 32. All applicable comparators pass the full CPU-reference check. These tests supplement sampled campaign checks, without establishing full-output coverage of every archived configuration. The expanded test executable also passes Compute Sanitizer memcheck with zero reported errors. Racecheck restricted to the project’s row-owned and pointer-refresh kernels reports zero hazards. Library kernels are outside that racecheck filter. These checks do not substitute for sanitizer coverage of every timed configuration.

4.7 Startup, storage, and repeated products

On native inputs, each method records synchronized host startup time from its prepared host representation through construction, GPU transfer, library initialization or preprocessing, and one completed product. Common input and output buffers and the CUDA context already exist. Matrix storage is constructed separately for each method, and method order is randomized. This measures application-level startup within an existing CUDA process, not cold driver initialization. Correctness work is excluded.

Persistent storage counts the explicitly allocated device arrays, including matrix values and indices, row classifications, pointer and offset arrays, and the workspace requested through cuSPARSE. It excludes common dense panels, host-side metadata, allocator overhead, and opaque library-internal allocations. In particular, grouped cuBLAS storage is not presented as a measurement of its handle’s internal memory use.

For k products, we report the cost model $T_m(k) = S_m + (k - 1)H_m$, where S_m is startup including the first product and H_m is the synchronized host median for subsequent calls. A second model adds measured host geometry and representation assembly. Direct, BSR8, and grouped methods start from packed blocks, while compact CSR and the dense control use the separately timed scalar assembly. Dense expansion is included in its startup observation. The preparation script records one CPU assembly observation per matrix, not a distribution. These are amortization estimates for serial synchronized calls, not timed solver runs, CUDA-graph execution, or asynchronous throughput measurements. For each k , the comparator is the lowest total among all measured library methods. No tuning cost is charged to that lower envelope.

4.8 Aggregation and uncertainty

For headline configuration c , matrix seed $s \in S = \{2, 3, 5\}$, process label p , comparator b , and direct method d , let

$$y_{c,s,p} = \log(t_{b,c,s,p}/t_{d,c,s,p}), \quad g_c = \exp\left(\frac{1}{3|S|} \sum_{s \in S} \sum_{p=1}^3 y_{c,s,p}\right).$$

Here t is a process median, and the best-library comparator takes the minimum applicable library median within that same process. The overall geometric mean is $G = \exp(|\mathcal{C}|^{-1} \sum_{c \in \mathcal{C}} \log g_c)$. Each configuration, seed, and process has equal weight. Ratios above one favor direct execution. The headline win count uses $g_c > 1$, not individual timing trials. The repeatability table additionally reports per-seed wins, computed over three processes for each configuration and seed. For the earlier seed-1 campaign, $S = \{1\}$, so the formula reduces to three-process averaging without matrix-seed variation.

We also report the range of three aggregate ratios obtained by holding the process label fixed, averaging over seeds where the campaign includes them. This is a descriptive repeatability check, not a confidence interval or a test of significance. Three process observations provide limited information about tails and shared thermal drift. No population-level confidence interval is assigned to this deliberately chosen grid, and no comparison of interval widths is interpreted as a decomposition of uncertainty. Matrix-instance variation, workload selection, process noise, and hardware variation remain distinct questions.

5 Results

5.1 Choosing direct execution

Table 2 compares the complete core grid by panel width, including MAGMA. Each width contains four shape distributions, both localities, and four degrees. The MAGMA column selects the faster of its slot and reduction compositions within each process. The best-library column selects the fastest compatible method, including BSR32 on uniform blocks. These are comparator-favoring lower envelopes, not deployed tuning policies. Section 5.2 gives this campaign’s protocol, its repeatability, and the transport trace immediately after. Campaigns run under earlier protocols are reported in the supplement and are excluded from every aggregate here.

Direct execution has a geometric-mean ratio of 1.139 against the fastest library and 2.003 against the faster MAGMA composition. It wins 105 of 128 best-library comparisons. All four widths have aggregate advantages, while RHS 64 wins all 32 individual comparisons. The narrow-width exceptions remain visible in the table.

Retaining packed blocks also avoids scalar metadata expansion and fixed-tile conversion. Section 5.5 measures the corresponding storage and startup costs. These matter when a matrix is replaced before many products can amortize a library conversion.

Table 2: Choosing an implementation on the 4,096-block-row core, matrix seeds 2, 3, and 5. Ratios divide comparator time by direct time and average in log space over three processes per seed. Each process uses twenty samples per method and single-product synchronization.

	RHS	BSR8	MAGMA best	Best library	Direct wins
	8	1.083	2.385	1.060	27/32
	16	1.090	2.098	1.045	24/32
	32	1.128	1.593	1.088	22/32
	64	1.446	2.019	1.398	32/32

5.2 MAGMA, transport traces, and controlled factors

The focused comparison adds the FP32 variable-size batched GEMM dispatcher from MAGMA revision `c3de65f855dd` [12]. One composition follows Boukaram et al.: each batch contains at most one block per output row, and successive slots accumulate sequentially. A second composition batches all block products into temporary storage, then reduces them by output row. The latter tests whether avoiding sequential slots offsets its workspace and reduction costs. Both cache dimensions and pointers, borrow device values, and use the upstream GEMM dispatcher without kernel changes. Setup validates dimensions before calling the dispatcher used by MAGMA’s `max_nocheck` interface. Its normal launch and attribute-setting overhead remains inside timing.

The headline and queued comparisons follow Table 1. Eight-product samples reuse the same input and output buffers. Methods and jobs are randomized, with all GPU processes run serially. The comparator set includes the explicit CSR algorithms, grouped cuBLAS, BSR8, applicable BSR32, and both MAGMA compositions. As noted above, only the 64 boundary of the related hierarchical study’s leaf sizes falls inside the supported dimensions, so this comparison measures the compatible block product rather than that study’s larger leaves or its construction algorithm.

Repeatability. The headline and width plot use single-product timings. A configuration ratio averages paired log ratios equally over three seeds and three processes per seed. Table 3 retains both timing modes. Its wins use the three-process geometric mean for each configuration and seed. Seed ranges average over processes, and process-label ranges average over seeds. These descriptive ranges are not confidence intervals. Repeating both factors avoids using timing trials as independent matrix samples.

An aggregate range can hide a configuration whose winner changes between observations, which matters most at the narrow widths where the aggregate advantage is a few percent. The last three columns therefore describe individual configurations rather than width averages. A configuration counts as a win beyond the margin when its own nine observations have a geometric mean above 1.05, which is a practical threshold for changing an implementation. A configuration is unanimous when all nine observations favor direct, and split when some favor direct and others favor a library.

Table 3: Repeated MAGMA-inclusive comparisons. Each width and batch contains 32 configurations, three seeds, and three processes per seed. Ratios above one favor direct. Wins count 96 configuration and seed combinations. The last three columns count individual configurations out of 32: wins with a geometric mean above 1.05, configurations where all nine observations favor direct, and configurations whose winner changes between observations.

	RHS	Batch	Best library	Seed range	Process range	Wins	Wins > 5%	Unanimous	Split
	8	1	1.060	1.057 to 1.063	1.057 to 1.064	81/96	23/32	24/32	7/32
	16	1	1.045	1.043 to 1.049	1.043 to 1.047	72/96	22/32	24/32	0/32
	32	1	1.088	1.084 to 1.090	1.087 to 1.088	68/96	20/32	20/32	4/32
	64	1	1.398	1.396 to 1.401	1.397 to 1.399	96/96	32/32	32/32	0/32
	8	8	1.052	1.048 to 1.060	1.048 to 1.060	71/96	21/32	21/32	5/32
	16	8	1.032	1.031 to 1.035	1.031 to 1.033	72/96	21/32	24/32	0/32
	32	8	1.079	1.076 to 1.082	1.078 to 1.079	63/96	20/32	20/32	3/32
	64	8	1.395	1.391 to 1.397	1.394 to 1.397	96/96	32/32	32/32	0/32

The overall ratios are 1.139 for individual products and 1.131 for batches of eight. The corresponding configuration win counts, averaged across both seeds and processes, are 105 and 101 out of 128.

Applying the 5 percent margin to single products leaves 97 of 128 configurations, and 100 are unanimous. The two results separate by width. At RHS 32 and 64, 52 of 64 configurations clear the margin and every RHS-64 configuration is unanimous. At RHS 8 and 16, 45 of 64 clear it and 7 change winner between observations. The few-percent aggregates at those widths therefore describe a population that includes configurations a deployment would not move, and some whose winner is not stable across repetitions. The narrow-width results support a measured description of one device, not a tuned dispatch rule.

An application trace. A standalone periodic transport example solves $u_t + u_x = 0$ using semi-Lagrangian discontinuous Galerkin projection in a Legendre modal basis [14]. Prescribed element orders repeat 7, 15, 31, and 63, giving block sizes 8, 16, 32, and 64. Each step translates the field by one quarter cell along exact characteristics, then projects it onto the receiving element basis. The self and upwind-neighbor overlap integrals define the two dense blocks in each row. Their structure comes directly from the discretization.

All six cases reach the same physical time $T = 1/128$, with $\Delta t = h/4$. Element counts 256, 1,024, and 4,096 therefore take 8, 32, and 128 dependent products, respectively. Ensemble widths are 8 and 64. Each case has three fresh processes and six trace samples per method. A sample includes the entire alternating-buffer trace, an initial-state reset, and output poisoning. All methods use the same assembled FP32 matrix. Every final coefficient is compared with repeated double-precision sparse products. The actual GPU output also passes a physical L^2 comparison with the analytically translated waves, with relative tolerance 2×10^{-3} . The executable must reject unchanged input before measuring any method. Preparation requires more than a tenfold rejection margin on both coefficient error criteria. A separate FP64 spatial-refinement study at fixed physical time checks the discretization without the FP32 arithmetic floor.

The full overlap equations, refinement observations, and startup definition are in the supplement. This remains a standalone transport component with prescribed orders. What it does demonstrate is that the measured advantage survives three conditions the synthetic cases do not impose. The block structure comes from the discretization rather than from a chosen partition, each product consumes the previous output so no step can be reordered or overlapped, and the timed trace produces coefficients that pass both the double-precision and the analytic comparison.

Table 4: Complete transport traces to $T = 1/128$. Ratios divide comparator time by direct time and first average paired process ratios in log space. The best-library column includes both MAGMA compositions and all tested NVIDIA controls. Times are direct GPU milliseconds per trace.

Elements	RHS	Steps	Direct ms	MAGMA best	Best library	Wins
256	8	8	0.108	2.290	0.840	1/3
256	64	8	0.223	2.069	1.065	3/3
1,024	8	32	0.681	3.934	1.186	3/3
1,024	64	32	2.392	2.085	1.244	3/3
4,096	8	128	12.697	2.971	1.220	3/3
4,096	64	128	50.824	1.833	1.439	3/3

On the 4,096-element, RHS-64 transport case, the fastest-library-to-direct ratio is 1.439, with 3 of three direct wins. The ratio against the faster MAGMA composition is 1.833. The table retains smaller traces and narrow ensembles, including cases that favor another implementation. This is evidence for the stated dependent transport product, not a universal application speedup.

Two specialized transport comparators. The comparators above are general implementations, so they do not answer whether a transport code should call this kernel at all. Repeating element orders and a constant translation give the operator only 8 distinct dense blocks, whatever the element count, and preparation already caches them. A specialized implementation would exploit that. We built two, differing in what they give up, and measured both under the protocol above. Both run the same dependent trace, in FP32, and pass the same coefficient and analytic checks.

The batched comparator recovers the distinct operators from the packed matrix by deduplicating block payloads, groups the elements that share an operator, and runs each group as one batched SGEMM whose operator pointer array repeats that single cached block. Both panels are bound when the plan is built, so every pointer array is uploaded once and none is refreshed during the trace. It issues 10 batched calls per product and never assembles the matrix on the device. At 4,096 elements one plan keeps 33.8 KiB of operator values and 225.8 KiB once its batch pointer arrays are counted. The alternating trace holds one plan per direction, each owning its own copies, so it retains 451.5 KiB against the 33.8 MiB packed matrix, a factor of

76.5. Dense panels are common to the methods and excluded from these figures. Internal library allocations are also excluded. Direct and fused execution do not use cuBLAS.

The fused comparator combines operator caching with fused accumulation. Its kernels mirror the release kernels for the panel width they serve, with the same thread mapping, shared input staging, register accumulators and single output write. One thread array owns an element, accumulates both contributions before writing, and reads the cached operator for its order class instead of its own packed copy. Because the order schedule repeats with period four, it derives the block structure from the element index and stores no per-element metadata at all, keeping only 33.8 KiB of operators for any element count. That specialization is valid only for this operator, so its constructor checks the assumed periodicity, degree, columns and payload equality against the packed matrix and refuses anything else.

Table 5: Both specialized transport comparators against direct execution on the same six traces, with the matched control. Batched, Fused and Copies each divide comparator time by direct time, so values above one favor direct execution and values below one favor the comparator. Copies is the fused kernel reading a private operator copy per element. Sharing divides Copies by Fused, isolating what operator sharing is worth, and its range gives the smallest and largest of three paired process ratios. Ratios and absolute times are geometric means across three processes, each using the median of six timed traces. Times are GPU milliseconds per trace.

Elements	RHS	Steps	Direct ms	Batched	Fused ms	Fused	Copies	Sharing	Sharing range
256	8	8	0.114	5.318	0.089	0.779	0.839	1.077	1.024 to 1.189
256	64	8	0.220	3.548	0.217	0.985	0.961	0.976	0.950 to 0.991
1,024	8	32	0.676	4.066	0.667	0.986	0.984	0.998	0.973 to 1.012
1,024	64	32	2.422	1.784	2.284	0.943	0.944	1.001	0.998 to 1.003
4,096	8	128	13.022	1.367	5.797	0.445	0.975	2.190	2.170 to 2.203
4,096	64	128	51.763	1.702	38.082	0.736	1.026	1.395	1.391 to 1.398

Direct execution is faster than the batched comparator in every case, from 1.367 to 5.318 times, winning all 18 of 18 process comparisons. The fused comparator reverses that: it is faster than direct execution in every case, by 1.014 to 2.246 times, taking 17 of 18 process comparisons. The matched control separates its two advantages. Operator sharing accounts for 1.395 times at 4,096 elements and RHS 64 and 2.190 times at RHS 8, while at 256 and 1,024 elements it accounts for between 0.976 and 1.077 times, showing more modest differences than at 4,096 elements.

Across the six configurations, the fused comparator has geometric speedup 1.274, including 1.359 on the headline RHS-64 trace.

The fused comparator changes four things at once relative to direct execution: which operator bytes an element reads, whether per-element metadata is loaded, how addresses are computed, and the order of the two contributions. A matched control separates the first from the rest. It is the same kernel, thread mapping, addressing arithmetic and contribution order, with one change: each element reads a private copy of its operators instead of the shared class copy, so the operand bytes are duplicated exactly as the assembled matrix duplicates them. The Copies column gives that variant against direct execution, and the Sharing column divides it by the fused time, which isolates what operator sharing is worth.

The largest observed sharing benefits occur at 4,096 elements, where it is worth 1.395 at RHS 64 and 2.190 at RHS 8, and the duplicated variant falls back to roughly direct execution’s time (1.026 and 0.975). At 256 and 1,024 elements the same control gives between 0.976 and 1.077, showing more modest differences than at 4,096 elements. These observations do not establish equivalence or exclude an operator-sharing effect.

The operand-source control is consistent with better cache reuse, but memory traffic was not measured, so cache residency remains an interpretation. Direct execution beats the fastest general library in five of six configurations and the specialized batched-GEMM plan in all six after process averaging. It reaches 45 to 99 percent of the fused comparator’s throughput, and 74 percent on the headline trace. The fused kernel is preferable for the repeated operator it supports. The general plan supports the full block-format contract without application-specific kernel code.

Separating workload factors. A crossed control varies block rows over 128, 512, degree over 4, 16, 128, block size over 8, 32, and panel width over 8, 64. Each of the 24 configurations uses seed 11 and three fresh processes. Degrees select nested prefixes of the same per-row column permutation, and shape changes preserve that adjacency. The same library controls are retained, with BSR32 added where compatible. Row-count changes also change the working set, so they are not interpreted as isolated occupancy measurements. The

supplementary table gives every controlled combination. These experiments distinguish the chosen factors without assigning a single cause retrospectively to the covariance result. Averaging equally over the other crossed factors, increasing degree from 4 to 128 changes the BSR8-to-direct ratio from 0.959 to 0.528 at RHS 8, and from 0.925 to 0.578 at RHS 64. Changing block size from 8 to 32 increases that ratio by factors of 2.014 and 3.099, respectively. The row-count contrast is much smaller at RHS 8 and unfavorable at RHS 64. Thus both degree and shape affect the performance boundary, and more block rows alone do not guarantee an advantage.

5.3 Fixed blocks, scaling, and degree imbalance

The remaining subsections cover the workloads outside the core grid, plus the fixed-block control. Both partitions have constant extent in uniform matrices, making fixed-block BSR a suitable control. Across the 32 uniform configurations of the seed-1 campaign, the BSR ratio is 0.989, with 10 direct wins. Table 6 shows the width dependence and helps distinguish variable-block reuse from the benefits already available with a uniform block format.

Table 6: Fixed-block BSR divided by direct execution on the uniform-32 subset. Each width contains eight degree and locality configurations.

RHS	BSR ratio	Minimum	Wins
8	0.982	0.951	2/8
16	0.884	0.823	0/8
32	0.925	0.903	0/8
64	1.192	1.112	8/8

The supplementary scaling table compares constant degree and the imbalanced pattern at three sizes. Each row includes both high-variance and bimodal dimensions and all four panel widths. The workload changes affect both total work and load balance. These results therefore test sensitivity to the combined changes and do not isolate the cost of imbalance at a fixed nonzero count. The tested maximum row degree remains 16, so heavy-tailed application degrees are not covered by this synthetic extension.

5.4 Application-derived inputs

Table 7 compares direct execution with compact CSR on the artificially partitioned published matrices. Compact CSR has an aggregate time ratio of 0.352, with 0 direct wins across twelve matrix and width configurations. Direct execution beats cached grouped cuBLAS on the same packed inputs by 5.246 in aggregate. The block-fill column shows how much of the packed storage contains source nonzeros.

Table 7: Compact-CSR lower-envelope time divided by packed direct time for published matrices under the stated artificial partitions. A ratio below one favors compact CSR. Each cell averages three process observations.

Matrix	Block fill	RHS 8	RHS 16	RHS 32	RHS 64
bcsstk13	8.9%	0.223	0.210	0.220	0.311
bcsstk14	12.2%	0.509	0.382	0.548	0.590
bcsstk15	6.9%	0.320	0.315	0.415	0.433

Low block fill makes the direct method perform arithmetic on internal zeros that compact CSR omits. These cases test the cost of choosing a packed representation for scalar sparse input. An application should retain compact CSR when its source has scattered nonzeros rather than dense blocks. Figure 3 shows these comparisons alongside uniform BSR, where the original block structure is already fixed.

5.5 Native near-field and lifecycle results

Table 8 describes the geometry-derived partitions. The clustered cases contain variable leaf sizes and much higher degrees than the synthetic grid, while the uniform cases provide small-leaf controls. The table reports overall scalar density as well as block shape. Dense SGEMM provides a useful alternative for the denser cases. All comparisons compute the identical near-field component.

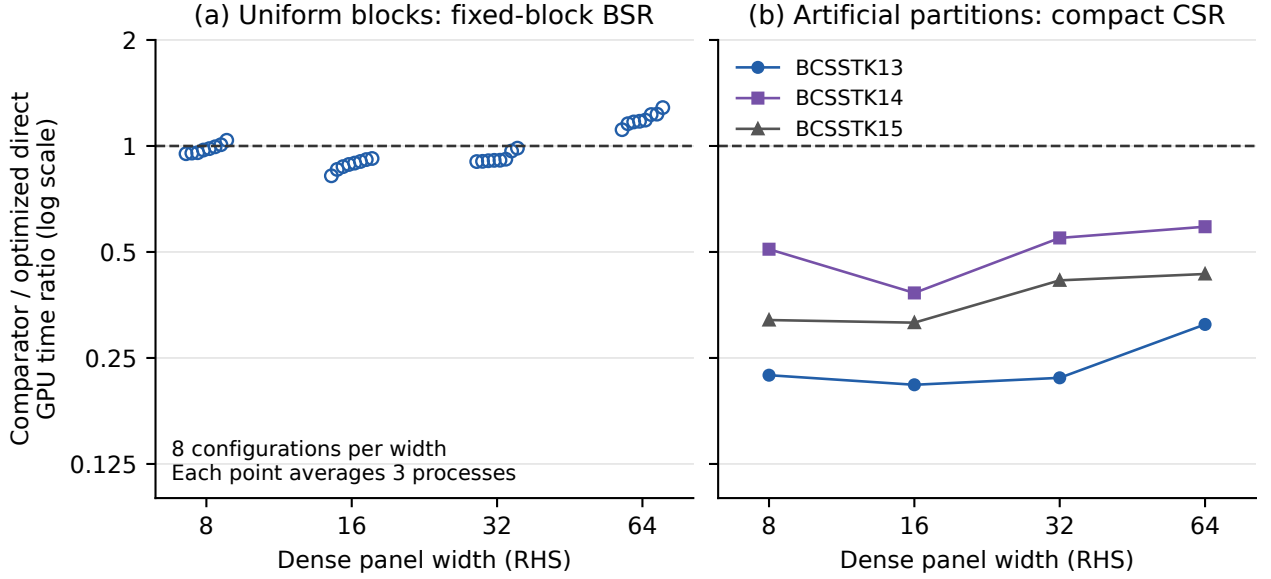


Figure 3: Format controls with a shared logarithmic ratio scale. Each point is the geometric mean of three paired process ratios. Left: all 32 uniform configurations, with eight degree and locality combinations per width. Horizontal offsets separate the points and have no numerical meaning. Right: the compact-CSR lower envelope for the twelve application configurations. Lines connect widths of the same matrix. Ratios below the dashed parity line favor the comparator. The panels represent different workloads and should not be read as a paired comparison between BSR and compact CSR.

Table 8: Native-model shape and degree ranges across three geometry seeds.

Points	Geometry	Leaf sizes	Block rows	Mean degree	Density %
2048	uniform	8	256 to 256	122.5 to 127.4	47.8 to 49.8
2048	clustered	8, 16, 32, 64	159 to 167	136.7 to 141.8	88.5 to 89.8
8192	uniform	8, 16	920 to 932	178.0 to 181.7	20.0 to 20.1
8192	clustered	8, 16, 32, 64	454 to 465	219.1 to 226.2	70.1 to 71.1

The aggregate BSR8 ratio on this component is 0.390, so BSR8 is faster for steady-state execution here. Direct execution offers a different benefit in startup, measured below. These matrices have many more blocks per row than the synthetic grid. Their comparison measures the cost of retaining whole block rows under that workload. Geometry, row degree, and leaf size vary together, so the two geometry families do not isolate their individual effects.

Table 9: Native-input cost models. Ratios divide the best library total by the direct total, averaging equally across input, width, and process. The assembly column adds one recorded CPU preparation observation per matrix and representation.

Products	Prepared-host ratio	Including assembly ratio
1	3.953	0.531
10	2.668	0.544
100	0.824	0.615
1000	0.412	0.491

Starting from prepared host representations, direct startup has a geometric ratio of 3.953 against the fastest library alternative, including one completed product. The modeled ten-product ratio is 2.668. This measures the adoption cost when packed blocks already exist. The second column of Table 9 adds CPU assembly and therefore answers a different question about constructing the input representations.

The most compact measured scalar-CSR plan uses 1.948 times the explicit device storage of the direct representation, averaged geometrically over native inputs and widths. BSR8 and grouped GEMM retain block storage and have different metadata costs. The host assembly routines are reference implementations and have not been optimized against each other.

Table 10: Explicit device-array storage ranges over native inputs and panel widths, and geometric mean synchronized startup time including one product. Storage excludes dense panels and opaque library allocations.

Method	Min MiB	Max MiB	Startup ms
direct	8.02	183.13	6.413
compact_alg1_pre	15.35	364.63	28.311
compact_alg2	15.64	371.50	40.899
compact_alg3_pre	15.33	363.99	65.078
bsr8	7.78	184.79	34.679
grouped_cached	8.85	185.84	37.324
dense	16.00	256.00	194.317

5.6 Matched mechanism results

The full-panel staging rows compare global and shared input loads with the same 64-thread mapping the narrow kernel uses, so they separate the contribution of staging from a change in thread count. The 256-thread factorial controls and their full table are in the supplement.

Table 11: Previous narrow-policy time divided by final direct time, measured in the same process. Each row includes two degrees, two shape distributions, three matrix seeds, and three processes per seed. Ranges first average the three processes for each matrix instance.

RHS	Block rows	Ratio	Min	Max
8	256	1.565	1.034	1.859
8	1024	2.019	1.677	2.385
16	256	1.846	1.301	2.356
16	1024	1.699	1.392	1.999

The paired narrow-policy speedups are 1.777 at RHS 8 and 1.771 at RHS 16 across this grid, on seeds 1, 2, and 3. Both policies use the same input, timing protocol, and full-output reference. The tuning evidence behind the choice is in the supplement.

6 Scope and limitations

Measurements cover one GPU and one local operating environment. Three process observations describe repeatability over this campaign, while long-term drift and hardware variation require separate measurements. The campaign designs differ and are not interchangeable. The MAGMA-inclusive campaign crosses three matrix seeds with three processes per seed in both timing modes. The matched and native experiments each use three generated instances with three processes per instance, so their ranges cover input variation as well as process noise. The transport campaigns, both the main one and the specialized comparison, instead run six configurations of three element counts and two panel widths, with one deterministic input per configuration and three processes on it. Different element counts are different configurations, not repeated draws of one workload, so transport ranges describe process variation only. These chosen workloads do not define a random application population. Campaigns run under earlier protocols are reported separately in the supplement and are excluded from these averages.

The synthetic extension tests one degree-imbalance pattern. The published matrices have artificial partitions, and the covariance component uses eight-point partition granularity to satisfy the block-size contract. Its startup and storage results cover a generated component rather than a complete solver or hierarchical construction pipeline. Total storage and setup still include matrix values and classification metadata.

No measured packed-block workload was captured from an independent application. The transport program was written for this evaluation, and its repeated operators permit the fused comparator to outperform direct execution. A captured workload or integration into an existing application would test adoption relevance. The motivating hierarchical study also uses leaves above our maximum dimension of 64, so its larger blocks and construction pipeline remain untested.

Performance evidence covers one GPU. Narrow-panel gains are small and vary by configuration, and the dispatch thresholds have not been tested on another architecture. Wider-panel gains are larger on this GPU but also lack cross-architecture validation.

Library lower envelopes favor the comparators by selecting measured medians without charging a tuning cost. The grouped cuBLAS baseline is the stated slot composition. MAGMA is measured with both slots and a separate reduction. Tensor-core research kernels are outside the FP32 comparison scope. Other reordering methods, other block ensembles, and additional precisions are outside this evaluation. The work contributes an implementation and measured execution policy for an established variable-block representation, with performance claims confined to the reported comparisons.

7 Reproducibility

The public API also accepts a non-owning `DeviceMatrix` when constructing a plan. Construction waits for the supplied stream and validates a host copy of metadata. Values remain on the device. Callers retain allocation ownership and keep structure fixed while the plan is in use. Device kernels may update values in place, ordered before execution. An owning matrix can instead enqueue a device-to-device value update without invalidating its plans. Structural changes require a new validated plan. Value updates and structure revalidation are measured separately, and the prepared-host startup ratios do not extend to these input paths.

The verified Windows reference workflow uses CUDA 13.4 and MSVC 19.44, building host and device sources as C++20. Each archive carries the compiler settings its own measurements used. From the repository root, the retained evidence and PDFs are rebuilt with:

```
uv sync --locked
uv run --locked powershell -File scripts/build_paper.ps1
```

The build validates the archives and regenerates tables before compiling both PDFs. The README gives a complete fresh-output workflow, including MAGMA preparation, the comparison build, all six transport inputs, and collection under the required relevance subdirectory. The comparison also has standalone commands in `scripts/build_relevance.py` and `scripts/run_relevance.py`. Its MAGMA revision, transport input recipe, and commands are documented in `docs/relevance.md`. Python dependencies are pinned in `pyproject.toml` and `uv.lock`. Paper building also uses pdfLaTeX and pdftotext.

The specialized transport comparison is a separate campaign, run by `scripts/run_dg.py` into `data/dg/` and exported by `scripts/export_dg.py`. It reuses the same transport inputs, horizon, and numerical checks, and it records the discovered operator count, batched call count, and stored bytes before any timing, so a record that does not report the structure the comparator exploits is rejected. Its comparator plan refuses to construct when the input lacks repeated block operators, which keeps it from standing in for a general product.

The canonical archive is `data/relevance-v2/`. Its checksummed `inputs.zip`, stored in checked 64 MiB parts, retains every transport matrix, initial state, numerical and analytic reference, mass-weight vector, metadata file, and refinement observation. Validation requires complete input coverage, payload hashes, expected panel extents, and agreement between commands and physical metadata. It also requires the executable's unchanged-state rejection and all GPU analytic checks. Superseded archives are listed in the supplement and generate no current result.

`scripts/build_verified.py` hashes all compiled inputs before a clean build, checks them again afterward, runs CTest, and records the resulting executable hashes and build commands. A cached receipt is reused only when every source and executable hash still matches. Each campaign snapshots the source and includes this receipt, and every process records its executable checksum. The analyzers verify the receipt against the source archive and reject missing or mismatched commands, matrix identities, design coverage, method counts, trial indices, nonfinite timings, and unsuccessful processes. The hashes verify internal consistency of the archived artifacts.

Final measurements are under `data/product-evaluation/`, in the `synthetic/`, `published/`, `native/`, and `ablation/` subdirectories, with the additional seeds and queued timings under `data/robustness/`. Generated tables and figures come from these campaigns. Native geometry, partition metadata, assembly observations, and input checksums are retained under `data/native/`. Large native binary arrays are regenerated by the preparation script and verified against their recorded checksums. Regeneration preserves the archived assembly observations. Use a fresh preparation directory to measure assembly again.

`scripts/analyze_revision.py` validates the main library campaigns. `scripts/analyze_supplement.py` validates native and matched controls. Both emit full-precision summaries beside the generated tables. `scripts/build_paper.ps1` requires both analyses to pass before compiling the manuscript. After compilation it extracts the rendered text of both documents and runs `scripts/check_references.py`, which rejects unresolved references and control sequences that reached the page as literal text. A successful compilation

does not catch either defect. Regression tests in `tests/test_evidence.py` exercise corrupt timing records and stale source or binary receipts.

8 Conclusion

VarBlockSpMM provides direct execution of packed variable blocks with one output writer and no execution workspace. The MAGMA-inclusive core gives 1.139 over the fastest tested library, with 97 of 128 configurations exceeding a 5 percent margin. The large RHS-64 transport trace gives 1.439 over the fastest library, while its fused application-specific comparator is 1.359 faster than direct execution. BSR8 remains preferable for covariance steady state and compact CSR for the published scalar inputs. These results support choosing the implementation by representation, shape, panel width, and reuse. The device-input and value-update APIs let callers retain packed allocations within the supported format. Independent application adoption and performance portability remain unestablished.

References

- [1] C. Yang, A. Buluç, and J. D. Owens. Design Principles for Sparse Matrix Multiplication on the GPU. *Euro-Par*, 2018. <https://arxiv.org/abs/1803.08601>
- [2] P. Okanovic et al. High Performance Unstructured SpMM Computation Using Tensor Cores. 2024. <https://arxiv.org/abs/2408.11551>
- [3] NVIDIA. cuSPARSE API reference, SpMM algorithms and preprocessing. Accessed 5 September 2026. <https://docs.nvidia.com/cuda/cusparse/index.html#cusparsespm>
- [4] NVIDIA. cuBLAS grouped batched GEMM. Accessed 5 September 2026. <https://docs.nvidia.com/cuda/cublas/#cublas-t-gemmgrouppedbatched>
- [5] W. Boukaram, Y. Liu, P. Ghysels, and X. S. Li. Adaptive Sketching Based Construction of H^2 Matrices on GPUs. *IPDPSW*, 2025. <https://arxiv.org/abs/2506.16759>
- [6] R. Vuduc and H.-J. Moon. Fast Sparse Matrix-Vector Multiplication by Exploiting Variable Block Structure. *HPCC*, 2005. https://doi.org/10.1007/11557654_91
- [7] R. Vuduc, J. W. Demmel, and K. A. Yelick. An interface for a self-optimizing sparse matrix kernel library. OSKI design document, draft, 14 March 2005, Section 5.2, p. 30. <https://bebop.cs.berkeley.edu/oski/oski-interface.pdf>
- [8] G. Huang, G. Dai, Y. Wang, and H. Yang. GE-SpMM: General-Purpose Sparse Matrix-Matrix Multiplication on GPUs for Graph Neural Networks. *SC20*, 2020. <https://doi.org/10.1109/SC41405.2020.00076>
- [9] NVIDIA. Asynchronous Data Copies. CUDA Programming Guide. Accessed 5 September 2026. <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/async-copies.html>
- [10] SuiteSparse Matrix Collection. HB/BCSSTK13, HB/BCSSTK14, and HB/BCSSTK15. Accessed 5 September 2026. <https://sparse.tamu.edu/HB/bcsstk13>
- [11] NIST Matrix Market. BCS Structural Engineering Matrices. BCSSTK13, BCSSTK14, and BCSSTK15 problem descriptions, in the Harwell-Boeing `bcsstruc1` and `bcsstruc2` sets. <https://math.nist.gov/MatrixMarket/data/Harwell-Boeing/>
- [12] MAGMA developers. Variable-size batched FP32 GEMM. Revision `c3de65f855dd`, accessed 6 September 2026. <https://github.com/icl-utk-edu/magma>
- [13] B. Cockburn and C.-W. Shu. Runge-Kutta Discontinuous Galerkin Methods for Convection-Dominated Problems. *Journal of Scientific Computing*, 16, 2001, pages 173 to 261. <https://www.dam.brown.edu/people/shu/pub.html>
- [14] O. Bokanowski and G. Simarmata. Semi-Lagrangian discontinuous Galerkin schemes for some first- and second-order partial differential equations. *ESAIM: Mathematical Modelling and Numerical Analysis*, 50(6), 2016, pages 1699 to 1730. <https://doi.org/10.1051/m2an/2016004>
- [15] NVIDIA. CUDA Compiler Driver NVCC, flush-to-zero behavior of `--use_fast_math` and `-ftz`. Accessed 10 September 2026. <https://docs.nvidia.com/cuda/cuda-compiler-driver-nvcc/>